



Extracting and Visualizing Stock Data

Description

Extracting essential data from a dataset and displaying it is a necessary part of data science; therefore individuals can make correct decisions based on the data. In this assignment, you will extract some stock data, you will then display this data in a graph.

Table of Contents

- Define a Function that Makes a Graph
- Question 1: Use yfinance to Extract Stock Data
- Question 2: Use Webscraping to Extract Tesla Revenue Data
- Question 3: Use yfinance to Extract Stock Data
- Question 4: Use Webscraping to Extract GME Revenue Data
- Question 5: Plot Tesla Stock Graph
- Question 6: Plot GameStop Stock Graph

Estimated Time Needed: **30 min**

***Note*:-** If you are working Locally using anaconda, please uncomment the following code and execute it.

```
In [6]: #!pip install yfinance==0.2.38  
#!pip install pandas==2.2.2  
#!pip install nbformat
```

```
In [177... !pip install yfinance  
!pip install bs4  
!pip install pandoc
```

Requirement already satisfied: yfinance in c:\users\baray\anaconda3\lib\site-packages (0.2.52)

Requirement already satisfied: pandas>=1.3.0 in c:\users\baray\anaconda3\lib\site-packages (from yfinance) (2.2.2)

Requirement already satisfied: numpy>=1.16.5 in c:\users\baray\anaconda3\lib\site-packages (from yfinance) (1.26.4)

Requirement already satisfied: requests>=2.31 in c:\users\baray\anaconda3\lib\site-packages (from yfinance) (2.32.3)

Requirement already satisfied: multitasking>=0.0.7 in c:\users\baray\anaconda3\lib\site-packages (from yfinance) (0.0.11)

Requirement already satisfied: lxml>=4.9.1 in c:\users\baray\anaconda3\lib\site-packages (from yfinance) (5.2.1)

Requirement already satisfied: platformdirs>=2.0.0 in c:\users\baray\anaconda3\lib\site-packages (from yfinance) (3.10.0)

Requirement already satisfied: pytz>=2022.5 in c:\users\baray\anaconda3\lib\site-packages (from yfinance) (2024.1)

Requirement already satisfied: frozendict>=2.3.4 in c:\users\baray\anaconda3\lib\site-packages (from yfinance) (2.4.2)

Requirement already satisfied: peewee>=3.16.2 in c:\users\baray\anaconda3\lib\site-packages (from yfinance) (3.17.8)

Requirement already satisfied: beautifulsoup4>=4.11.1 in c:\users\baray\anaconda3\lib\site-packages (from yfinance) (4.12.3)

Requirement already satisfied: html5lib>=1.1 in c:\users\baray\anaconda3\lib\site-packages (from yfinance) (1.1)

Requirement already satisfied: soupsieve>1.2 in c:\users\baray\anaconda3\lib\site-packages (from beautifulsoup4>=4.11.1->yfinance) (2.5)

Requirement already satisfied: six>=1.9 in c:\users\baray\anaconda3\lib\site-packages (from html5lib>=1.1->yfinance) (1.16.0)

Requirement already satisfied: webencodings in c:\users\baray\anaconda3\lib\site-packages (from html5lib>=1.1->yfinance) (0.5.1)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\baray\anaconda3\lib\site-packages (from pandas>=1.3.0->yfinance) (2.9.0.post0)

Requirement already satisfied: tzdata>=2022.7 in c:\users\baray\anaconda3\lib\site-packages (from pandas>=1.3.0->yfinance) (2023.3)

Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\baray\anaconda3\lib\site-packages (from requests>=2.31->yfinance) (3.3.2)

Requirement already satisfied: idna<4,>=2.5 in c:\users\baray\anaconda3\lib\site-packages (from requests>=2.31->yfinance) (3.7)

Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\baray\anaconda3\lib\site-packages (from requests>=2.31->yfinance) (2.2.3)

Requirement already satisfied: certifi>=2017.4.17 in c:\users\baray\anaconda3\lib\site-packages (from requests>=2.31->yfinance) (2024.12.14)

Requirement already satisfied: bs4 in c:\users\baray\anaconda3\lib\site-packages (0.0.2)

Requirement already satisfied: beautifulsoup4 in c:\users\baray\anaconda3\lib\site-packages (from bs4) (4.12.3)

Requirement already satisfied: soupsieve>1.2 in c:\users\baray\anaconda3\lib\site-packages (from beautifulsoup4->bs4) (2.5)

Collecting pandoc

 Downloading pandoc-2.4.tar.gz (34 kB)

 Preparing metadata (setup.py): started

 Preparing metadata (setup.py): finished with status 'done'

Collecting plumbum (from pandoc)

 Downloading plumbum-1.9.0-py3-none-any.whl.metadata (10 kB)

Requirement already satisfied: ply in c:\users\baray\anaconda3\lib\site-packages (from pandoc) (3.11)

```
Requirement already satisfied: pywin32 in c:\users\baray\anaconda3\lib\site-packages
(from plumbum->pandoc) (305.1)
Downloading plumbum-1.9.0-py3-none-any.whl (127 kB)
Building wheels for collected packages: pandoc
  Building wheel for pandoc (setup.py): started
  Building wheel for pandoc (setup.py): finished with status 'done'
  Created wheel for pandoc: filename=pandoc-2.4-py3-none-any.whl size=34821 sha256=6
cf3419a572e3fb7aab5d04dc7b1df1fdfb2cffaa1d813b36c5b74110bf3e853
  Stored in directory: c:\users\baray\appdata\local\pip\cache\wheels\9c\2f\9f\b1aac8
c3e74b4ee327dc8c6eac5128996f9eadf586e2c0ba67
Successfully built pandoc
Installing collected packages: plumbum, pandoc
Successfully installed pandoc-2.4 plumbum-1.9.0
```

```
In [257... import plotly.io as pio
pio.renderers.default='jupyterlab'
```

```
In [259... import yfinance as yf
import pandas as pd
import requests
from bs4 import BeautifulSoup
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

In Python, you can ignore warnings using the warnings module. You can use the filterwarnings function to filter or ignore specific warning messages or categories.

```
In [262... import warnings
# Ignore all warnings
warnings.filterwarnings("ignore", category=FutureWarning)
```

Define Graphing Function

In this section, we define the function `make_graph`. **You don't have to know how the function works, you should only care about the inputs. It takes a dataframe with stock data (dataframe must contain Date and Close columns), a dataframe with revenue data (dataframe must contain Date and Revenue columns), and the name of the stock.**

```
In [266... def make_graph(stock_data, revenue_data, stock):
    fig = make_subplots(rows=2, cols=1, shared_xaxes=True, subplot_titles=("Histori
stock_data_specific = stock_data[stock_data.Date <= '2021--06-14']
revenue_data_specific = revenue_data[revenue_data.Date <= '2021-04-30']
fig.add_trace(go.Scatter(x=pd.to_datetime(stock_data_specific.Date), y=stock_da
fig.add_trace(go.Scatter(x=pd.to_datetime(revenue_data_specific.Date), y=revenu
fig.update_xaxes(title_text="Date", row=1, col=1)
fig.update_xaxes(title_text="Date", row=2, col=1)
fig.update_yaxes(title_text="Price ($US)", row=1, col=1)
fig.update_yaxes(title_text="Revenue ($US Millions)", row=2, col=1)
fig.update_layout(showlegend=False,
height=900,
title=stock,
```

```
xaxis_rangeslider_visible=True)  
fig.show()
```

Use the `make_graph` function that we've already defined. You'll need to invoke it in questions 5 and 6 to display the graphs and create the dashboard.

Note: You don't need to redefine the function for plotting graphs anywhere else in this notebook; just use the existing function.

Question 1: Use `yfinance` to Extract Stock Data

Using the `Ticker` function enter the ticker symbol of the stock we want to extract data on to create a ticker object. The stock is Tesla and its ticker symbol is `TSLA`.

```
In [271... stock_data = yf.Ticker("TSLA")
```

Using the ticker object and the function `history` extract stock information and save it in a dataframe named `tesla_data`. Set the `period` parameter to `"max"` so we get information for the maximum amount of time.

```
In [274... tesla_data = stock_data.history(period = "max")
```

Reset the index using the `reset_index(inplace=True)` function on the `tesla_data` DataFrame and display the first five rows of the `tesla_data` dataframe using the `head` function. Take a screenshot of the results and code from the beginning of Question 1 to the results below.

```
In [277... tesla_data.reset_index(inplace = True)  
tesla_data.head(5)
```

Out[277...

	Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
0	2010-06-29 00:00:00-04:00	1.266667	1.666667	1.169333	1.592667	281494500	0.0	0.0
1	2010-06-30 00:00:00-04:00	1.719333	2.028000	1.553333	1.588667	257806500	0.0	0.0
2	2010-07-01 00:00:00-04:00	1.666667	1.728000	1.351333	1.464000	123282000	0.0	0.0
3	2010-07-02 00:00:00-04:00	1.533333	1.540000	1.247333	1.280000	77097000	0.0	0.0
4	2010-07-06 00:00:00-04:00	1.333333	1.333333	1.055333	1.074000	103003500	0.0	0.0

Question 2: Use Webscraping to Extract Tesla Revenue Data

Use the `requests` library to download the webpage <https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/revenue.htm> Save the text of the response as a variable named `html_data` .

In [281...

```
url="https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/revenue.htm"
html_data = requests.get(url).text
```

Parse the html data using `beautiful_soup` using parser i.e `html5lib` or `html.parser` . Make sure to use the `html_data` with the content parameter as follow `html_data.content` .

In [283...

```
soup=BeautifulSoup(html_data, "html.parser")
```

Using `BeautifulSoup` or the `read_html` function extract the table with `Tesla Revenue` and store it into a dataframe named `tesla_revenue` . The dataframe should have columns `Date` and `Revenue` .

- Step-by-step instructions
- Click here if you need help locating the table

```
In [289... table_revenue=soup.find_all("table")[1]
table_revenue

tesla_revenue=pd.DataFrame(columns=["Date", "Revenue"])

for row in table_revenue.find("tbody").find_all("tr"):
    col=row.find_all("td")
    date=col[0].text
    revenue=col[1].text

    tesla_revenue=pd.concat([tesla_revenue, pd.DataFrame({"Date":[date], "Revenue":
```

Execute the following line to remove the comma and dollar sign from the `Revenue` column.

```
In [292... tesla_revenue["Revenue"] = tesla_revenue['Revenue'].str.replace(r'[,,$]','', regex=
```

Execute the following lines to remove an null or empty strings in the Revenue column.

```
In [295... tesla_revenue.dropna(inplace=True)

tesla_revenue = tesla_revenue[tesla_revenue['Revenue'] != ""]
```

Display the last 5 row of the `tesla_revenue` dataframe using the `tail` function. Take a screenshot of the results.

```
In [298... tesla_revenue.tail(5)
```

```
Out[298...      Date  Revenue
48  2010-09-30      31
49  2010-06-30      28
50  2010-03-31      21
52  2009-09-30      46
53  2009-06-30      27
```

Question 3: Use yfinance to Extract Stock Data

Using the `Ticker` function enter the ticker symbol of the stock we want to extract data on to create a ticker object. The stock is GameStop and its ticker symbol is `GME`.

```
In [302... game_stock=yf.Ticker("GME")
```

Using the ticker object and the function `history` extract stock information and save it in a dataframe named `gme_data`. Set the `period` parameter to `"max"` so we get information for the maximum amount of time.

```
In [305... gme_data=game_stock.history(period="max")
```

Reset the index using the `reset_index(inplace=True)` function on the `gme_data` DataFrame and display the first five rows of the `gme_data` dataframe using the `head` function. Take a screenshot of the results and code from the beginning of Question 3 to the results below.

```
In [308... gme_data.reset_index(inplace=True)
gme_data.head(5)
```

```
Out[308...
```

	Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
0	2002-02-13 00:00:00-05:00	1.620128	1.693350	1.603296	1.691666	76216000	0.0	0.0
1	2002-02-14 00:00:00-05:00	1.712707	1.716073	1.670626	1.683250	11021600	0.0	0.0
2	2002-02-15 00:00:00-05:00	1.683250	1.687458	1.658002	1.674834	8389600	0.0	0.0
3	2002-02-19 00:00:00-05:00	1.666418	1.666418	1.578047	1.607504	7410400	0.0	0.0
4	2002-02-20 00:00:00-05:00	1.615920	1.662210	1.603296	1.662210	6892800	0.0	0.0

Question 4: Use Webscraping to Extract GME Revenue Data

Use the `requests` library to download the webpage <https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/stock.html>. Save the text of the response as a variable named `html_data_2`.

```
In [312... url="https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDvelope
html_data2=requests.get(url).text
```

Parse the html data using `beautiful_soup` using parser i.e `html5lib` or `html.parser`.

```
In [315... soup=BeautifulSoup(html_data2,"html.parser")
```

Using `BeautifulSoup` or the `read_html` function extract the table with `GameStop Revenue` and store it into a dataframe named `gme_revenue`. The dataframe should have columns `Date` and `Revenue`. Make sure the comma and dollar sign is removed from the `Revenue` column.

Note: Use the method similar to what you did in question 2.

► Click here if you need help locating the table

```
In [320... gme_revenue_data=soup.find_all("table")[1]

gme_revenue=pd.DataFrame(columns=["Date", "Revenue"])

for row in gme_revenue_data.find("tbody").find_all("tr"):
    col=row.find_all("td")
    date=col[0].text
    revenue=col[1].text
    gme_revenue=pd.concat([gme_revenue, pd.DataFrame({"Date":[date], "Revenue":[rev
gme_revenue["Revenue"]=gme_revenue["Revenue"].str.replace(",","").str.replace("$",
gme_revenue.dropna(inplace=True)
```

Display the last five rows of the `gme_revenue` dataframe using the `tail` function. Take a screenshot of the results.

```
In [323... gme_revenue.tail(5)
```

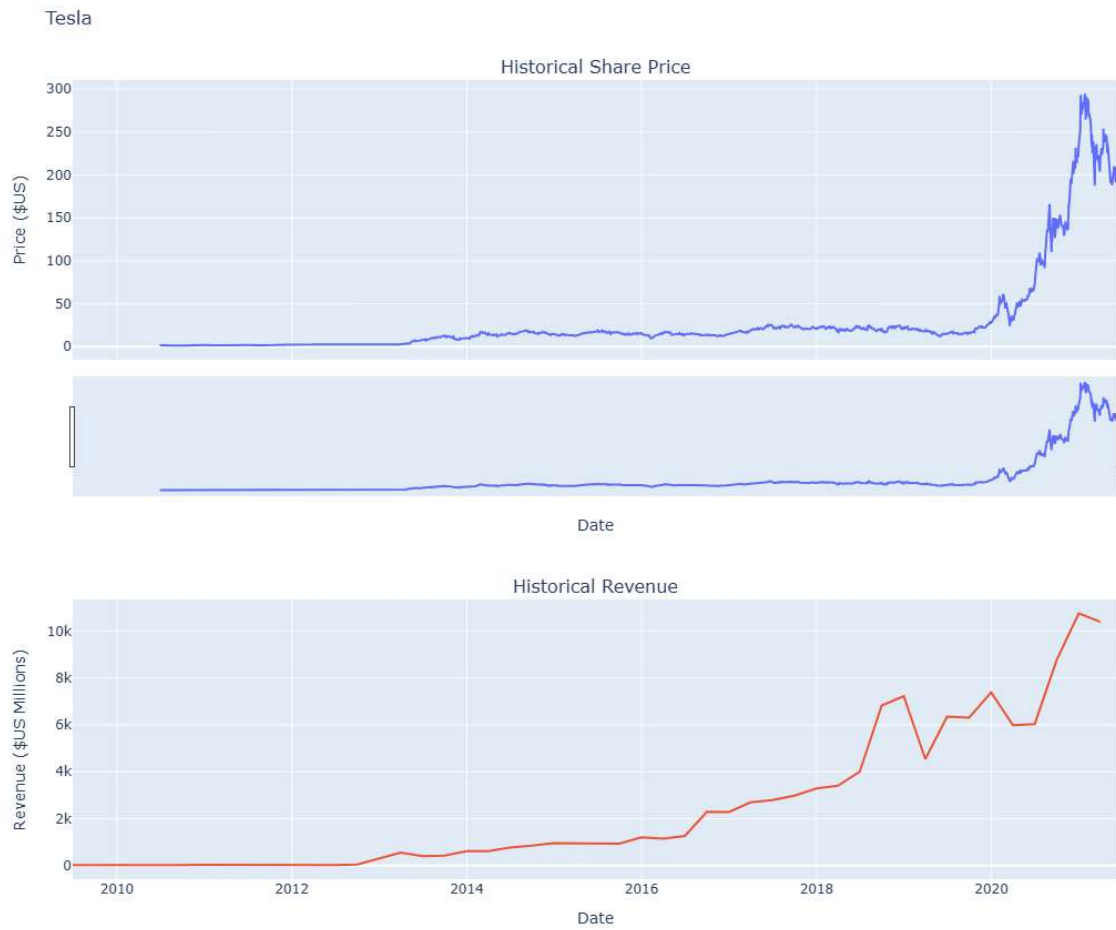
```
Out[323...
      Date  Revenue
57 2006-01-31    1667
58 2005-10-31     534
59 2005-07-31     416
60 2005-04-30     475
61 2005-01-31     709
```

Question 5: Plot Tesla Stock Graph

Use the `make_graph` function to graph the Tesla Stock Data, also provide a title for the graph. Note the graph will only show data upto June 2021.

► Hint

```
In [328... make_graph(tesla_data, tesla_revenue, "Tesla")
```

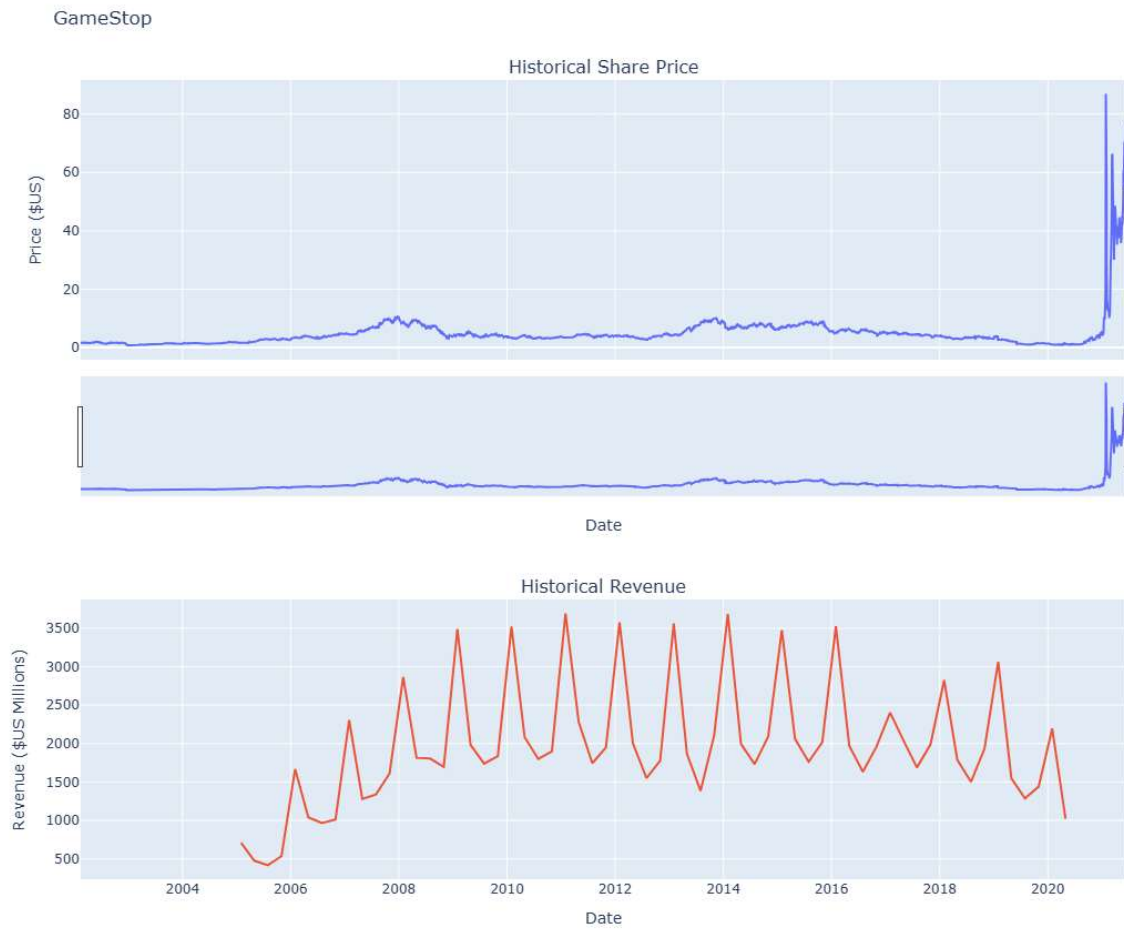



Question 6: Plot GameStop Stock Graph

Use the `make_graph` function to graph the GameStop Stock Data, also provide a title for the graph. The structure to call the `make_graph` function is `make_graph(gme_data, gme_revenue, 'GameStop')`. Note the graph will only show data upto June 2021.

► Hint

```
In [333... make_graph(gme_data, gme_revenue, "GameStop")
```



About the Authors:

[Joseph Santarcangelo](#) has a PhD in Electrical Engineering, his research focused on using machine learning, signal processing, and computer vision to determine how videos impact human cognition. Joseph has been working for IBM since he completed his PhD.

© IBM Corporation 2020. All rights reserved.

toggle

toggle

toggle

toggle